Explanation of Recursion and Stack in DFS

Recursion in Depth-First Search

Recursion is a way that a function can call itself to solve a problem. In Depth-First Search we use recursion to look at a graph by going into each part before moving on to the next.

In Depth-First Search we start at a node say we have visited it. Then we visit all the nodes next to it that we have not visited yet. Each time we call the function it gets added to the computers list of things to do which helps us keep track of what to do

How it Works

1. Visit the node we start with
2. Say we have visited it
3. Call the Depth-First Search function for each node next to it that we have not visited
4. Keep doing this until we have visited all nodes
5. Go to the previous node when we can't visit any more

Example

If we start at node 0 we do this:

Depth-First Search of 0 then 1 then 2.

When we get to the node we go back step by step.

Advantages

- Easy to write the code
- The code is simple and easy to read

Disadvantages

- Uses computer memory
- Can cause problems with graphs

Stack in Depth-First Search

A stack is a list that works like a Last In First Out system. We can also do Depth-First Search with a stack of recursion.

In this way we make our own list to keep track of what nodes to visit. The list has nodes that we need to look at.

How it Works

1. Add the starting node to the list
2. Take a node from the list
3. If we have not visited it:
    a. Say we have visited it
    b. Print it
4. Add all the nodes next to it that we have not visited to the list
5. Keep doing this until the list's empty

Example

Add 0 to the list.

Take 0. Visit it.

Add 1 and 2 to the list.

Take 2. Visit it.

Advantages

- Does not use recursion
- No risk of computer list getting too full
- Gives us control

Disadvantages

- The code is more complicated
- We have to make our list